Parul® University
Vadodara, Gujarat

NAAC GRADE A++

Information and Communication Technology

UNIT-5
Model fitting in R

Study Guide

Ms.Pallavi Sakalley
Assistant Professor
CSE, PIT
Parul University

- Regression Models
- Linear and Logistic Model
- Classification Models
- Decision Tree
- Naïve Bayes
- SVM
- Random Forest,
- Clustering Models – K Means and Hierarchical clustering

INDEX

Regression Models

Regression analysis is a statistical tool to estimate the relationship between two or more variables. There is always one response variable and one or more predictor variables. Regression analysis is widely used to fit the data accordingly and further, predicting the data for forecasting. It helps businesses and organizations to learn about the behavior of their product in the market using the dependent/response variable and independent/predictor variable.

Types of Regression in R:

There are mainly three types of Regression in R programming that is widely used. They are:

Linear Regression
Multiple Regression
Logistic Regression

Linear Regression

The Linear Regression model is one of the widely used among three of the regression types. In linear regression, the relationship is estimated between two variables i.e., one response variable and one predictor variable. Linear regression produces a straight line on the graph. Mathematically

$y = ax + b$

Where,

x indicates predictor or independent variable
y indicates response or dependent variable
a and **b** are coefficients

Implementation in R

In R programming, **lm()** function is used to create linear regression model.

Syntax: lm(formula)

Parameter:

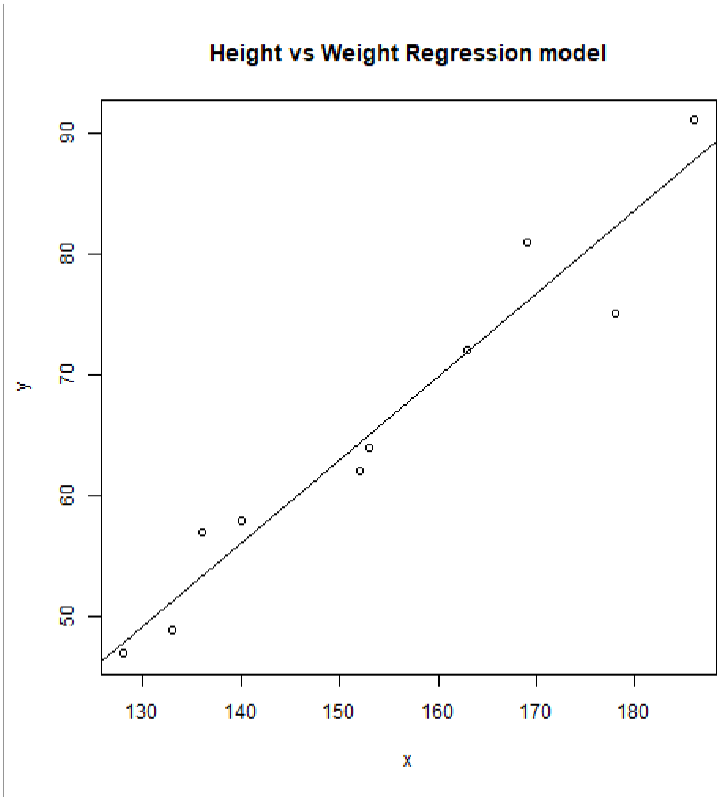
formula: represents the formula on which data has to be fitted To know about more optional parameters, use below command in console: help("lm")
Example: In this example, let us plot the linear regression line on the graph and predict the weight-based using height.

```
# R program to illustrate # Linear Regression


 #  Height vector
 x  <- c(153, 169, 140, 186,          128,

    136, 178, 163, 152,               133)



# Weight vector

y <- c(64, 81, 58, 91, 47, 57,

75, 72, 62, 49)



# Create a linear regression model model <- lm(y~x)


# Print regression model print(model)


# Find the weight of a person # With height 182
df <- data.frame(x = 182) res <-  predict(model, df)
cat("\nPredicted value of a person

with height = 182")

print(res)



# Output to be present as PNG file
```

```
png(file = "linearRegGFG.png")


# Plot

plot(x, y, main = "Height vs Weight

Regression model")

abline(lm(y~x))


# Save the file. dev.off()
```

Output:

```
Call:
lm(formula = y ~ x)
Coefficients:
(Intercept)       x
-39.7137        0.6847
Predicted value of a person with height = 182 1
84.9098
```


Height vs Weight Regression model
y
x
50
60
70
80
90
130
140
150
160
170
180

Logistic Regression

Logistic Regression is another widely used regression analysis technique and predicts the value with a range. Moreover, it is used for predicting the values for categorical data. For example, Email is spam or non-spam, winner or loser, male or female, etc. Mathematically,

$$\frac{1}{1+e^{-z}}$$

where,
y represents response variable
z represents equation of independent variables or features

Implementation in R

In R programming, **glm()** function is used to create a logistic regression model.

Syntax: glm(formula, data, family)

Parameters:

formula: represents a formula on the basis of which model has to be fitted
data: represents dataframe on which formula has to be applied
family: represents the type of function to be used. "binomial" for logistic regression

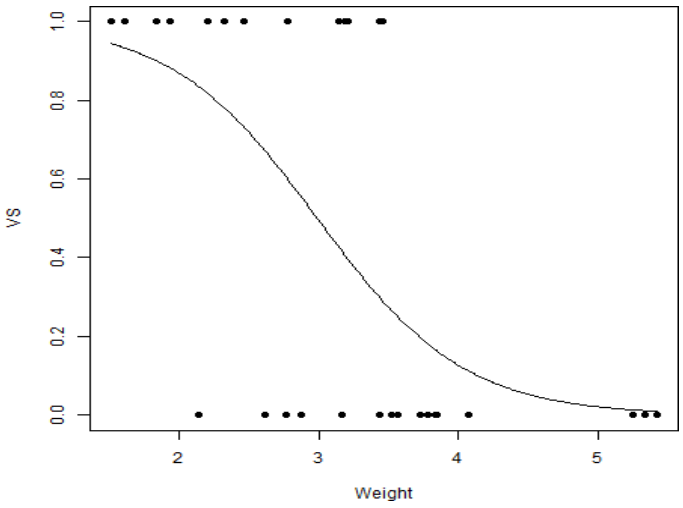
Example:

```
# R program to illustrate # Logistic Regression


# Using mtcars dataset

# To create the logistic model model <- glm(formula = vs ~ wt,
family = binomial, data = mtcars)


# Creating a range of wt values
```

```
<- seq(min(mtcars$wt),

max(mtcars$wt), 0.01)


# Predict using weight

<- predict(model, list(wt = x),

type = "response")



# Print model print(model)


# Output to be present as PNG file png(file = "LogRegGFG.png")


# Plot

plot(mtcars$wt, mtcars$vs, pch = 16, xlab = "Weight", ylab = "VS")
lines(x, y)



# Saving the file dev.off()
```

Output:

```
Call:  glm(formula = vs ~ wt, family = binomial, data = mtcars)

Coefficients:
(Intercept)        wt
5.715   -1.911
Degrees of Freedom: 31 Total (i.e. Null);  30 Residual Null Deviance:       43.86
Residual Deviance: 31.37        AIC: 35.37
```



Classification in R Programming

R is a very dynamic and versatile programming language for data science. This article deals with classification in R. **Various Classifiers are:**

Decision Trees
Naive Bayes Classifiers
K-NN Classifiers
Support Vector Machines(SVM's)

Decision Tree Classifier

It is basically is a graph to represent choices. The nodes or vertices in the graph represent an event and the edges of the graph represent the decision conditions. Its common use is in Machine Learning and Data Mining applications. **Applications:**
Spam/Non-spam classification of email, predicting of a tumor is cancerous or not. Usually, a model is constructed with noted data also called training dataset. Then a set of validation data is used to verify and improve the model. R has packages that are used to create and visualize decision trees.
The R package "party" is used to create decision trees.
Command:

```
install.packages("party")
```

```
# Load the party package. It will automatically load other # dependent packages.
```

```
library(party)

# Create the input data frame. input.data <-readingSkills[c(1:105), ]

# Give the chart file a name. png(file="decision_tree.png")

# Create the tree. output.tree <-ctree(
nativeSpeaker ~ age +shoeSize +score, data = input.dat)

# Plot the tree. plot(output.tree)

# Save the file. dev.off()
```

Output:

```
null device
1
Loading required package: methods Loading required package: grid Loading required package:
mvtnorm Loading required package: modeltools Loading required package: stats4 Loading required
package: strucchange Loading required package: zoo

Attaching package: 'zoo'
The following objects are masked from 'package:base': as.Date, as.Date.numeric
Loading required package: sandwich
```

Naive Bayes Classifier

Naïve Bayes classification is a general classification method that uses a probability approach, hence also known as a probabilistic approach based on Bayes' theorem with the assumption of independence between features. The model is trained on training dataset to make predictions by **predict()** function. **Formula:**

$$P(A|B)=P(B|A)\times P(A)P(B)$$

It is a sample method in machine learning methods but can be useful in some instances. The training is easy and fast that just requires considering each predictor in each class separately.

Application:
It is used generally in sentimental analysis.

```
library(caret)
## Warning: package 'caret' was built under R version 3.4.3 set.seed(7267166)
trainIndex =createDataPartition(mydata$prog, p =0.7)$Resample1 train = mydata[trainIndex, ]
test =mydata[-trainIndex, ]

## check the balance print(table(mydata$prog)) ##
## academic    general vocational ## 105       45        50
print(table(train$prog))
```

Output:

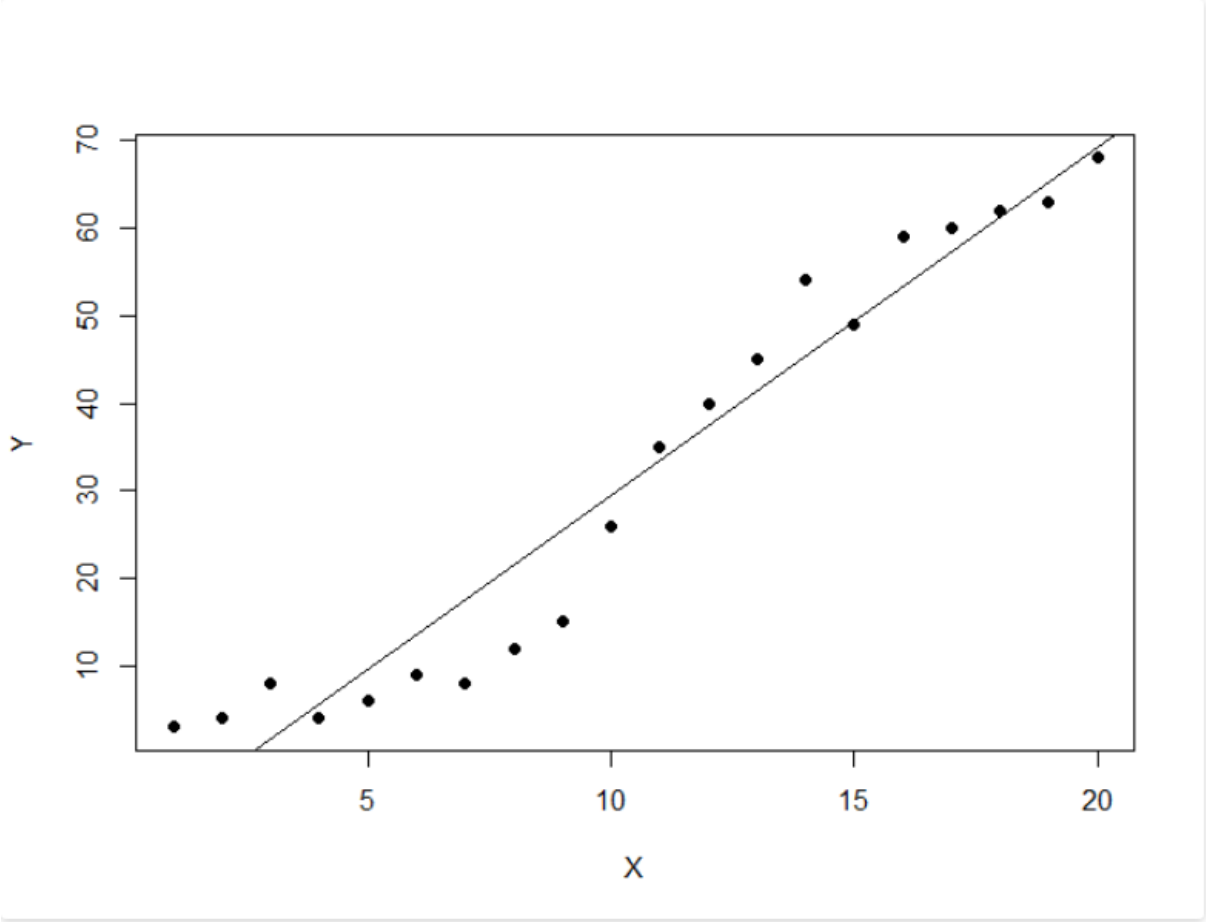
```
## Naive Bayes Classifier for Discrete Predictors ##
## Call:
## naiveBayes.default(x = X, y = Y, laplace = laplace) ##
## A-priori probabilities:
## Y
##   academic    general vocational ##  0.5248227  0.2269504  0.2482270 ##
## Conditional probabilities:
##        science
## Y    [, 1]     [, 2]
##        academic         54.21622 9.360761
##        general 52.18750 8.847954
##        vocational 47.31429 9.969871 ##
##        socst
## Y    [, 1]     [, 2]
##        academic         56.58108   9.635845
##        general 51.12500   8.377196
##        vocational 44.82857 10.279865
```

Support Vector Machines (SVM's)

A support vector machine (SVM) is a supervised binary machine learning algorithm that uses classification algorithms for two-group classification problems. After giving an SVM model sets of labeled training data for each category, they're able to categorize new text. Mainly SVM is used for text classification problems. It classifies the unseen data. It is widely used than Naive Bayes.SVM id usually a fast and dependable classification algorithm that performs very well with a limited amount of data. **Applications:**
SVMs have a number of applications in several fields like Bioinformatics, to classify genes, etc.

```
# Load the data from the csv file
dataDirectory <-"D:/" # put your own folder here
data <-read.csv(paste(dataDirectory, 'regression.csv', sep =""), header = TRUE)

# Plot the data plot(data, pch =16)

# Create a linear regression model model <-lm(Y ~ X, data)

# Add the fitted line abline(model)
```

Output:



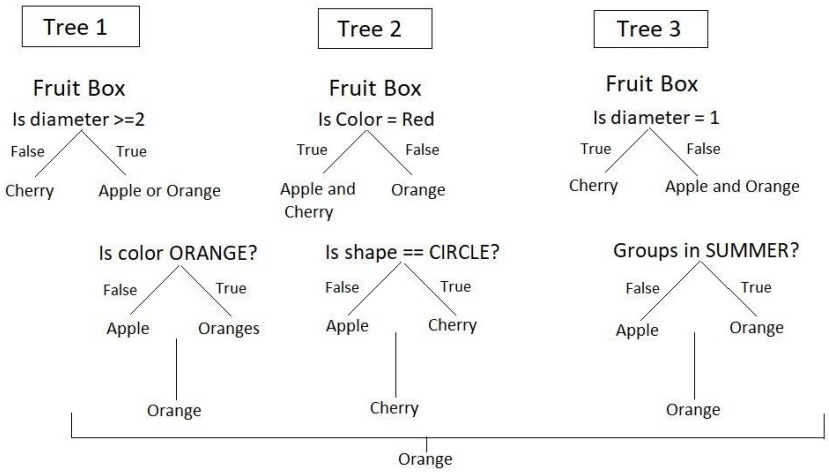
Random Forest Approach in R Programming

Random Forest in R Programming is an ensemble of decision trees. It builds and combines multiple decision trees to get more accurate predictions. It's a non-linear classification algorithm. Each decision tree model is used when employed on its own. An error estimate of cases is made that is not used when constructing the tree. This is called an out of bag error estimate mentioned as a percentage.

They are called **random** because they choose predictors randomly at a time of training. They are called **forest** because they take the output of multiple trees to make a decision. Random forest outperforms decision trees as a large number of uncorrelated trees(models) operating as a committee will always outperform the individual constituent models.

Theory

Random forest takes random samples from the observations, random initial variables(columns) and tries to build a model. Random forest algorithm is as follows:
Draw a random bootstrap sample of size **n** (randomly choose **n** samples from training data).
Grow a decision tree from bootstrap sample. At each node of tree, randomly select **d** features.
Split the node using features(variables) that provide best split according to objective function. For instance, by maximizing the information gain.
Repeat steps 1 to step 2, **k** times(k is the number of trees you want to create using subset of samples).
Aggregate the prediction by each tree for a new data point to assign the class label by majority vote i.e pick the group selected by most number of trees and assign new data point to that group.
Example:
Consider a Fruit Box consisting of three fruits Apples, Oranges, and Cherries in training data i.e n = 3. We are predicting the fruit which is maximum in number in a fruit box. A random forest model using the training data with a number of trees, k = 3.



The model is judged using various features of data i.e diameter, color, shape, and groups. Among orange, cheery, and orange, orange is selected to be maximum in fruit box by random forest.

The Dataset

Irisdataset consists of 50 samples from each of 3 species of Iris(Iris setosa, Iris virginica, Iris versicolor) and a multivariate dataset introduced by British statistician and biologist Ronald Fisher in his 1936 paper *The use of multiple measurements in taxonomic problems*. Four features were measured from each sample i.e length and width of the sepals and petals and based on the combination of these four features, Fisher developed a linear discriminant model to distinguish the species from each other.

```
# Installing package
install.packages("caTools")     # For sampling the dataset install.packages("randomForest")  # For implementing random forest algorithm

# Loading package library(caTools) library(randomForest)

# Splitting data in train and test data split <-sample.split(iris, SplitRatio =0.7) split

train <-subset(iris, split =="TRUE") test <-subset(iris, split =="FALSE")

# Fitting Random Forest to the train dataset set.seed(120)  # Setting seed
classifier_RF =randomForest(x =train[-5],
y =train$Species, ntree = 500)

classifier_RF

# Predicting the Test set results
y_pred =predict(classifier_RF, newdata =test[-5])

# Confusion Matrix
confusion_mtx =table(test[, 5], y_pred) confusion_mtx

# Plotting model plot(classifier_RF)

# Importance plot importance(classifier_RF)

# Variable importance plot varImpPlot(classifier_RF)
```

Output:

```
> classifier_RF

Call:
 randomForest(x = train[-5], y = train$Species, ntree = 500)
               Type of random forest: classification
                     Number of trees: 500
No. of variables tried at each split: 2

        OOB estimate of  error rate: 3.33%
Confusion matrix:
           setosa versicolor virginica class.error
setosa         30          0         0  0.00000000
versicolor      0         29         1  0.03333333
virginica       0          2        28  0.06666667
```

Clustering in R Programming

Clustering in R Programming Language is an unsupervised learning technique in which the data set is partitioned into several groups called clusters based on their similarity. Several clusters of data are produced after the segmentation of data. All the objects in a cluster share common characteristics. During data mining and analysis, clustering is used to find similar datasets.

Types of Clustering in R Programming

In R, there are numerous clustering algorithms to choose from. Here are a few of the most **popular clustering techniques in R:**

K-means clustering: it is a data-partitioning technique that seeks to assign each observation to the cluster with the closest mean after dividing the data into k clusters.

Hierarchical clustering: By repeatedly splitting or merging clusters according to their similarity, hierarchical clustering is a technique for creating a hierarchy of clusters.

DBSCAN clustering: it is a density-based technique that divides regions with lower densities and clusters together data points that are close to one another.
Spectral clustering: Spectral clustering is a technique that turns the clustering problem into a graph partitioning problem by using the eigenvectors of the similarity matrix.

Fuzzy clustering: Instead of allocating each data point to a single cluster, fuzzy clustering allows points to belong to numerous clusters with varying degrees of membership.

Density-based clustering: A class of techniques known as density-based clustering groups together data points based on density rather than distance.

Ensemble clustering: Ensemble clustering is a technique for enhancing clustering performance by combining several clustering methods or iterations of the same algorithm.
Each kind of clustering technique has its own advantages and disadvantages and is appropriate for various kinds of data and clustering issues. The qualities of the data and the objectives of the research will determine which clustering technique is best.

Applications of Clustering in R Programming Language

Marketing: In R programming, clustering is helpful for the marketing field. It helps in finding the market pattern and thus, helps in finding the likely buyers. Getting the interests of customers using clustering and showing the same product of their interest can increase the chance of buying the product.
Medical Science: In the medical field, there is a new invention of medicines and treatments on a daily basis. Sometimes, new species are also found by researchers and scientists. Their category can be easily found by using the clustering algorithm based on their similarities.
Games: A clustering algorithm can also be used to show the games to the user based on his interests.
Internet: An user browses a lot of websites based on his interest. Browsing history can be aggregated to perform clustering on it and based on clustering results, the profile of the user is generated.

Methods of Clustering
There are 2 types of clustering in R programming:

Hard clustering: In this type of clustering, the data point either belongs to the cluster totally or not and the data point is assigned to one cluster only. The algorithm used for hard clustering is k-means clustering.
Soft clustering: In soft clustering, the probability or likelihood of a data point is assigned in the clusters rather than putting each data point in a cluster. Each data point exists in all the clusters with some probability. The algorithm used for soft clustering is the fuzzy clustering method or soft k-means.

K-Means Clustering in R Programming language
K-Means is an iterative hard clustering technique that uses an unsupervised learning algorithm. In this, total numbers of clusters are pre-defined by the user and based on the similarity of each data point, the data points are clustered. This algorithm also finds out the centroid of the cluster.

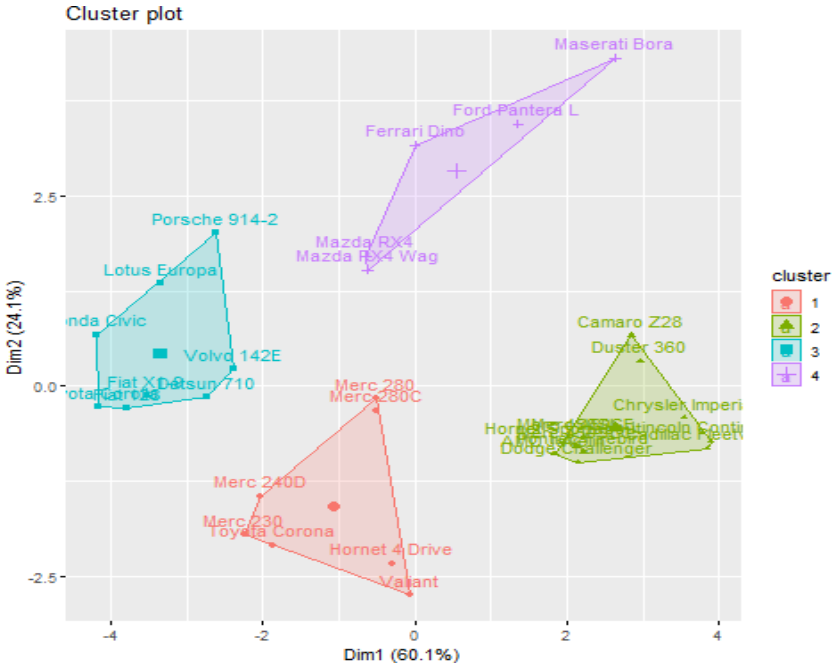
Algorithm:

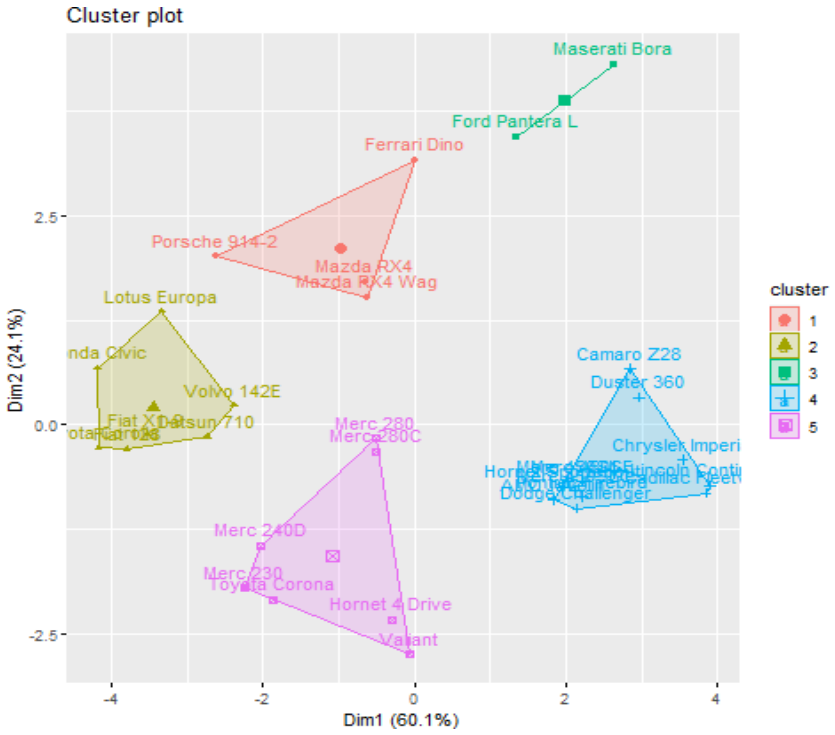
Specify number of clusters (K): Let us take an example of k =2 and 5 data points.
Randomly assign each data point to a cluster: In the below example, the red and green color shows 2 clusters with their respective random data points assigned to them.

Calculate cluster centroids: The cross mark represents the centroid of the corresponding cluster.
Re-allocate each data point to their nearest cluster centroid: Green data point is assigned to the red cluster as it is near to the centroid of red cluster.
Re-figure cluster centroid Syntax: kmeans(x, centers, nstart) **where,**
x represents numeric matrix or data frame object
centers represents the **K** value or distinct cluster centers
nstart represents number of random sets to be chosen

```
# Library required for fviz_cluster function install.packages("factoextra") library(factoextra)

# Loading dataset df <- mtcars

# Omitting any NA values df <- na.omit(df)

# Scaling dataset df <- scale(df)

# output to be present as PNG file png(file = "KMeansExample.png")

km <- kmeans(df, centers = 4, nstart = 25)

# Visualize the clusters fviz_cluster(km, data = df)

# saving the file dev.off()

# output to be present as PNG file png(file = "KMeansExample2.png")

km <- kmeans(df, centers = 5, nstart = 25)

# Visualize the clusters fviz_cluster(km, data = df)

# saving the file dev.off()
```

Output:
When k = 4





Using the fviz_cluster function from the factoextra package, this code applies k-means clustering to the mtcars dataset using two different values of centers (4 and 5) and then saves the cluster visualizations as PNG files.

The clustering technique uses the information from the mtcars dataset, which includes details on various automobile models including the number of cylinders, horsepower, and miles per gallon. The scale function is used to scale the data to have a zero mean and unit variance, and the na.omit function is used

to delete any rows with missing values.

Then, to improve the chance of discovering the global optimum, the means function is used to execute k-means clustering with 4 and 5 clusters. The fviz_cluster function, which plots the data points coloured by cluster membership and also displays the cluster centers, is then used to see the resulting cluster assignments.

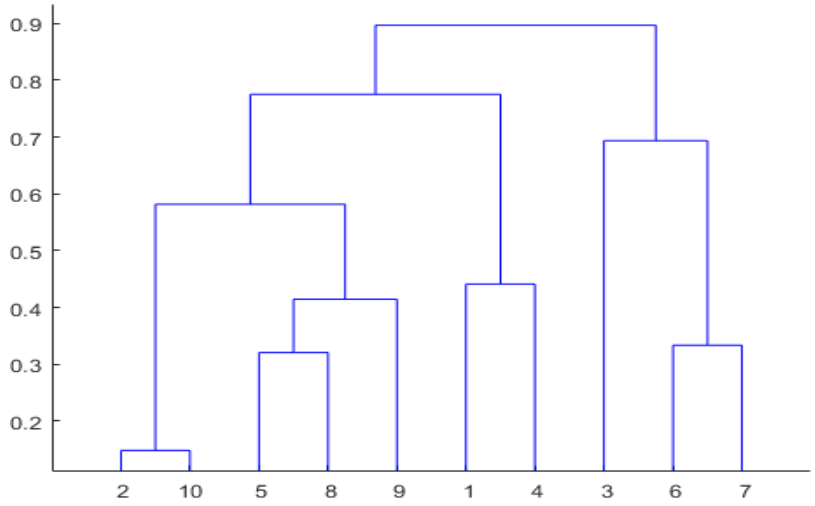
The png and dev. off functions are then used to save the generated plots as PNG files.
To save the files to the correct location on your computer, you might need to modify the file paths or file names used in the png function.
Hierarchical Clustering in R Programming
Hierarchical clustering in R Programming Language is an unsupervised non-linear algorithm in which clusters are created such that they have a hierarchy (or a pre-determined ordering). For example, consider a family of up to three generations. A grandfather and mother have their children that become father and mother of their children. So, they all are grouped together to the same family i.e they form a hierarchy.

R – Hierarchical Clustering

Hierarchical clustering is of two types:

Agglomerative Hierarchical clustering: It starts at individual leaves and successfully merges clusters together. Its a Bottom-up approach.
Divisive Hierarchical clustering: It starts at the root and recursively split the clusters. It's a top-down approach.

Theory:
In hierarchical clustering, Objects are categorized into a hierarchy similar to a tree-shaped structure which is used to interpret hierarchical clustering models. The algorithm is as follows:
Make each data point in a single point cluster that forms N clusters.
Take the two closest data points and make them one cluster that forms N-1 clusters.
Take the two closest clusters and make them one cluster that forms N-2 clusters.
Repeat steps 3 until there is only one cluster.



Dendrogram is a hierarchy of clusters in which distances are converted into heights. It clusters **n** units or objects each with **p** feature into smaller groups. Units in the same cluster are joined by a horizontal line. The leaves at the bottom represent individual units. It provides a visual representation of clusters. **Thumb Rule:** Largest vertical distance which doesn't cut any horizontal line defines the optimal number of clusters.

The Dataset
mtcars(motor trend car road test) comprise fuel consumption, performance, and 10 aspects of automobile design for 32 automobiles. It comes pre-installed with dplyr package in R.

Installing the package install.packages("dplyr")

Loading package library(dplyr)

Summary of dataset in package head(mtcars)

```
> head(mtcars)
                   mpg cyl disp  hp drat    wt  qsec vs am gear carb
Mazda RX4         21.0   6  160 110 3.90 2.620 16.46  0  1    4    4
Mazda RX4 Wag     21.0   6  160 110 3.90 2.875 17.02  0  1    4    4
Datsun 710        22.8   4  108  93 3.85 2.320 18.61  1  1    4    1
Hornet 4 Drive    21.4   6  258 110 3.08 3.215 19.44  1  0    3    1
Hornet Sportabout 18.7   8  360 175 3.15 3.440 17.02  0  0    3    2
Valiant           18.1   6  225 105 2.76 3.460 20.22  1  0    3    1
```

Performing Hierarchical clustering on Dataset
Using Hierarchical Clustering algorithm on the dataset using **hclust()** which is pre-installed in stats package when R is installed.

```
# Finding distance matrix
distance_mat <- dist(mtcars, method = 'euclidean') distance_mat

# Fitting Hierarchical clustering Model # to training dataset
set.seed(240)  # Setting seed
Hierar_cl <- hclust(distance_mat, method = "average")
Hierar_cl

# Plotting dendrogram plot(Hierar_cl)

# Choosing no. of clusters # Cutting tree by height
abline(h = 110, col = "green")

# Cutting tree by no. of clusters fit <- cutree(Hierar_cl, k = 3 ) fit

table(fit)
rect.hclust(Hierar_cl, k = 3, border = "green")
```

Output:

Distance matrix:

```
> distance_mat
                     Mazda RX4 Mazda RX4 Wag  Datsun 710 Hornet 4 Drive Hornet Sportabout
Mazda RX4 Wag        0.6153251
Datsun 710          54.9086059    54.8915169
Hornet 4 Drive      98.1125212    98.0958939 150.9935191
Hornet Sportabout  210.3374396   210.3358546 265.0831615    121.0297564
Valiant             65.4717710    65.4392224 117.7547018     33.5508692       152.1241352
Duster 360         241.4076490   241.4088680 294.4790230    169.4299647        70.1767262
Merc 240D           50.1532711    50.1146059  49.6584796    121.2739722       241.5069657
Merc 230            25.4683117    25.3284509  33.1803843    118.2433145       233.4924012
Merc 280            15.3641921    15.2956865  66.9363534     91.4224033       199.3344960
Merc 280C           15.6724727    15.5837744  67.0261397     91.4612914       199.3406564
Merc 450SE         135.4307018   135.4254826 189.1954941     72.4964325        84.3888482
Merc 450SL         135.4014424   135.3960351 189.1631745     72.4313532        84.3683999
Merc 450SLC        135.4794674   135.4723157 189.2345426     72.5718466        84.4332423
Cadillac Fleetwood 326.3395903   326.3355070 381.0926242    234.4403876       116.2804201
Lincoln Continental 318.0469808  318.0429333 372.8012090    227.9726091       108.0624299
Chrysler Imperial  304.7203408   304.7169175 359.3014906    218.1548299        97.2049146
Fiat 128            93.2679950    93.2530993  40.9933763    184.9689734       302.0377212
Honda Civic        102.8307567   102.8238713  52.7704607    191.5518700       310.0324645
Toyota Corolla     100.6040368   100.5887588  47.6535017    192.6714187       309.5581776
Toyota Corona       42.3075233    42.2659224  12.9654743    138.5304725       252.3331988
Dodge Challenger   163.1150750   163.1134210 217.7795805     72.4403915        48.9838851
AMC Javelin        149.6047203   149.6014522 204.3188913     61.3601899        61.4274240
Camaro Z28         233.2228758   233.2248748 286.0049209    163.6632641        70.9665308
Pontiac Firebird   248.6780270   248.6762035 303.3583889    156.2240346        40.0052475
Fiat X1-9           92.5048389    92.4940020  39.8815148    184.4471198       301.5669483
Porsche 914-2       44.4033659    44.4073589  13.1357109    139.1579524       254.1452553
Lotus Europa        65.7328377    65.7362635  25.0948550    163.2367437       272.3582423
Ford Pantera L     245.4247064   245.4293785 297.2940489    180.1140339        89.5934049
Ferrari Dino        66.7661029    66.7764167  90.2415509    130.5523007       215.0673853
Maserati Bora      265.6454248   265.6491465 309.7718171    229.3419352       170.7094473
Volvo 142E          39.1894029    39.1626037  20.6939436    137.0363299       248.0063378
```

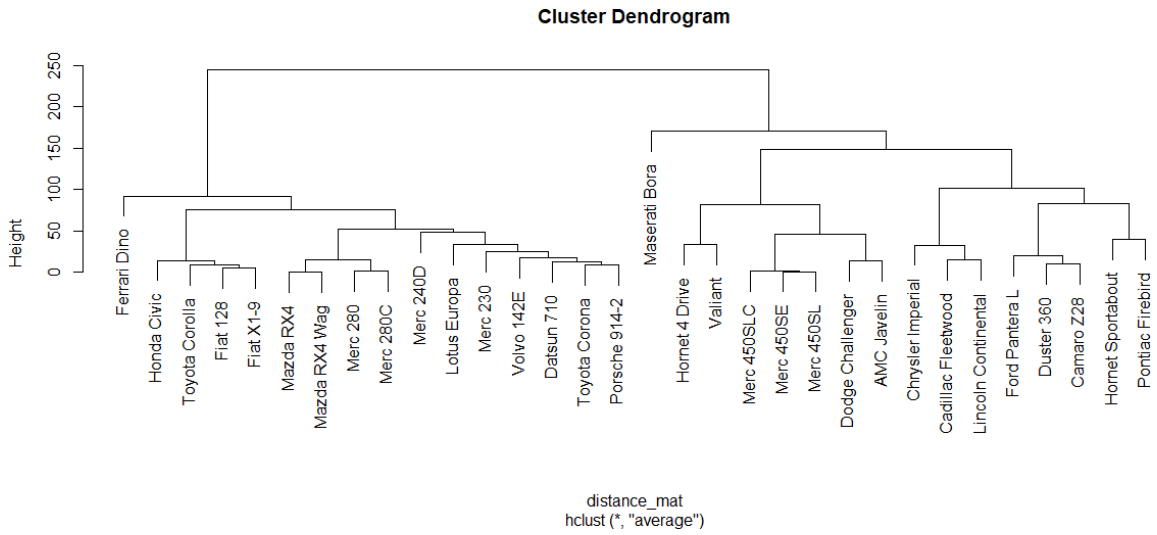
The values are shown as per the distance matrix calculation with the method as euclidean.

Model Hierar_cl:

```
> Hierar_cl

Call:
hclust(d = distance_mat, method = "average")

Cluster method   : average
Distance         : euclidean
Number of objects: 32
```

In the model, the cluster method is average, distance is euclidean and no. of objects are 32.

Plot dendrogram:

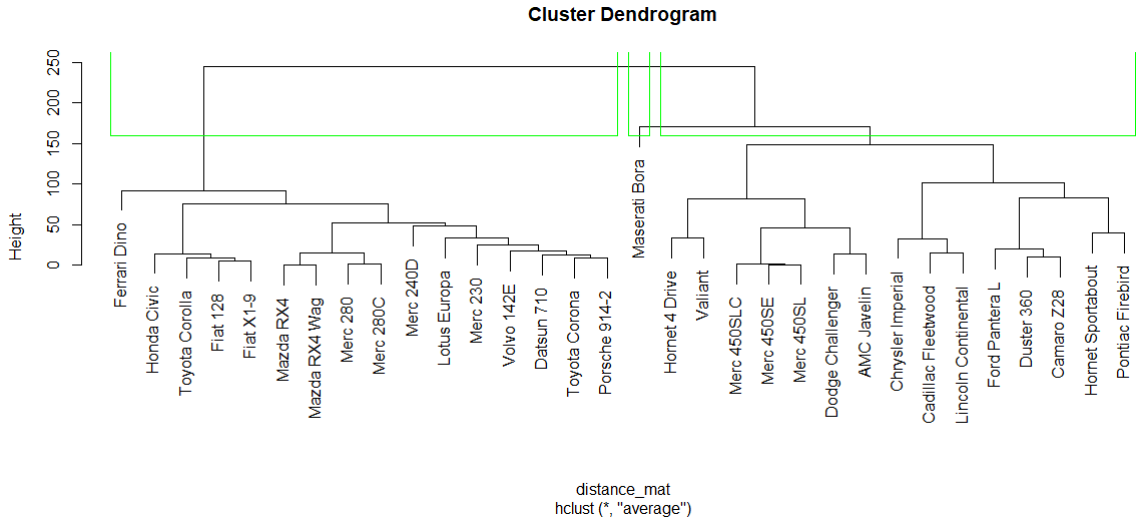


The plot dendrogram is shown with x-axis as distance matrix and y-axis as height.

Cutted tree:

```
> fit
          Mazda RX4       Mazda RX4 Wag          Datsun 710      Hornet 4 Drive
                  1                   1                   1                   2
  Hornet Sportabout             Valiant             Duster 360          Merc 240D
                  2                   2                   2                   1
           Merc 230            Merc 280            Merc 280C           Merc 450SE
                  1                   1                   1                   2
          Merc 450SL          Merc 450SLC   Cadillac Fleetwood Lincoln Continental
                  2                   2                   2                   2
  Chrysler Imperial            Fiat 128          Honda Civic      Toyota Corolla
                  2                   1                   1                   1
      Toyota Corona      Dodge Challenger          AMC Javelin          Camaro Z28
                  1                   2                   2                   2
   Pontiac Firebird            Fiat X1-9       Porsche 914-2         Lotus Europa
                  2                   1                   1                   1
     Ford Pantera L        Ferrari Dino       Maserati Bora          Volvo 142E
                  2                   1                   3                   1
```

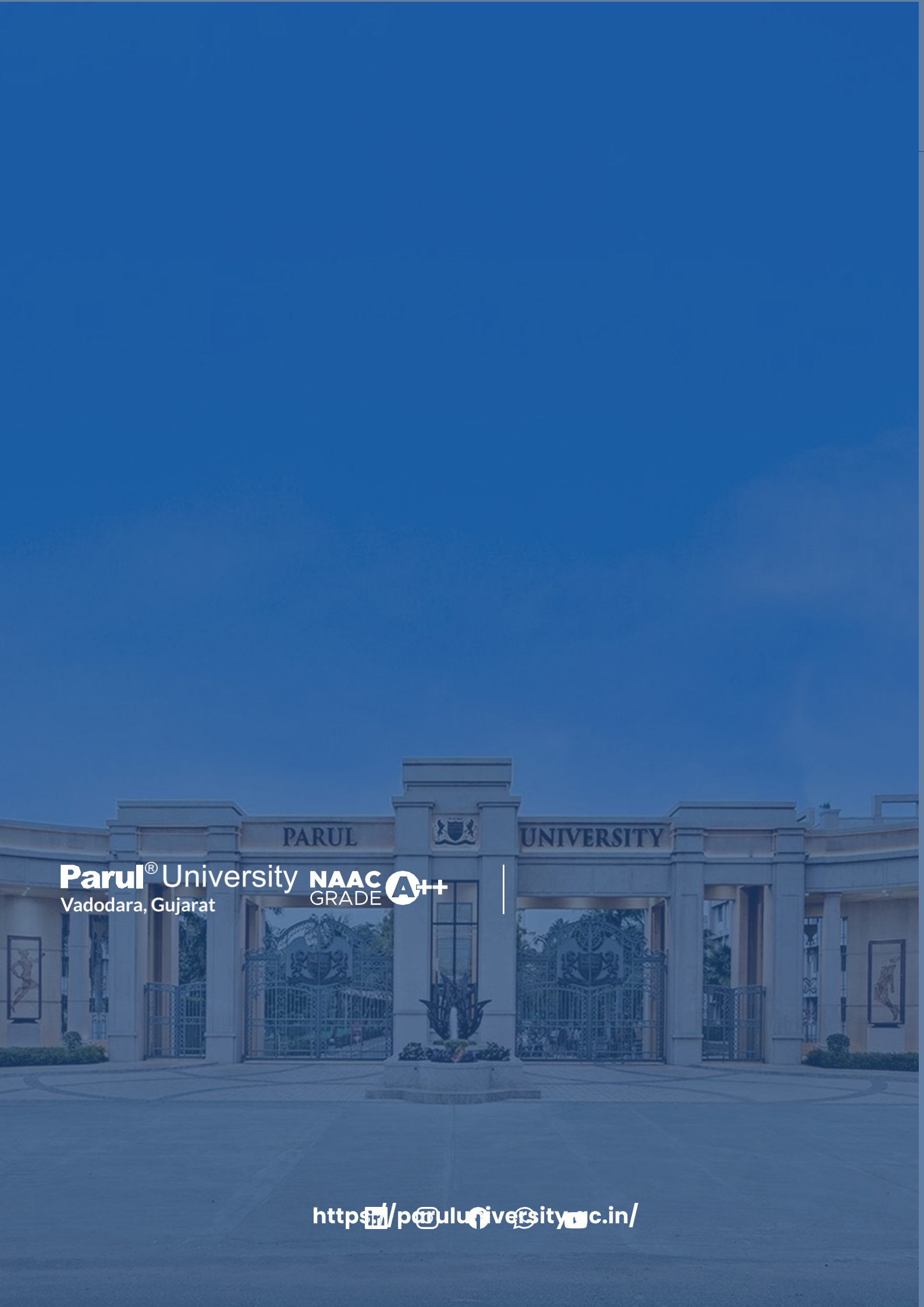
So, Tree is cut where k = 3 and each category represents its number of clusters.
Plotting dendrogram after cutting:



The plot denotes dendrogram after being cut. The green lines show the number of clusters as per the thumb rule.

References

1. Gareth James, Daniela Witten, Trevor Hastie, and Robert Tibshirani, *An Introduction to Statistical Learning with Applications in R*, Springer, 2021.
2. Brett Lantz, *Machine Learning with R*, 3rd Edition, Packt Publishing, 2019.
3. Luis Torgo, *Data Mining with R: Learning with Case Studies*, Chapman & Hall/CRC, 2016.
4. Brett Lantz, *Machine Learning in R*, Packt Publishing.
5. Nina Zumel and John Mount, *Practical Data Science with R*, Manning Publications, 2014.
6. Hadley Wickham and Garrett Grolemund, *R for Data Science*, O'Reilly Media, 2017.
7. GeeksforGeeks: https://www.geeksforgeeks.org/r-language/r-tutorial/


PARUL UNIVERSITY
Parul® University
Vadodara, Gujarat
NAAC GRADE A++
https://paruluniversity.ac.in/